DWH MCP EXPLORER

TECHNICAL IMPLEMENTATION GUIDE

Stage 6.9.2 | DWH MCP Explorer | English edition

Implementation baseline v1.0 · 7 November 2026

A code-oriented description of the validated architecture, MCP interface and data contracts. The document describes the implemented system, not planned capabilities.

1. Scope and design principles

A Python MCP server makes a fixed set of 21 tools available to an AI client. The server is the execution boundary; Claude Desktop requests operations and interprets structured responses.

The PostgreSQL + SQL repository track supplies relational metadata, SQL lineage and mapping reconciliation. The Azure Databricks track supplies Unity Catalog metadata, captured lineage events and exact grain validation.

Operations described here are metadata reads or read-only SQL. The Knowledge Base v1 is assembled on demand and is not a persistent indexed database.

Engineering principles

Boundary	Implementation rule
Thin adapter	server.py registers tools; implementation lives in dwh_mcp/ modules.
Testability	Composition functions accept providers as arguments; mocks can replace remote dependencies.
Evidence	Preserve event sources, counts, limitations and distinguish reads from confirmed dependency edges.
Safety	Validate SQL identifiers; store credentials outside source code; avoid mutation operations.

2. Runtime architecture and execution path

Discovery: Claude Desktop → local MCP connection → server.py tool registry (21 names).

PostgreSQL request: MCP Tool → metadata / SQL repository / lineage / reconciliation functions → structured result.

Databricks investigation: MCP Tool → metadata provider + lineage provider → on-demand Knowledge Object.

Grain check: MCP Tool → validated identifiers → SDK Statement Execution → SQL Warehouse SELECT → reconciliation result.

```
Claude Desktop (MCP client)
      |
      v
server.py  --  21 MCP tools
      |
      +-- PostgreSQL / SQL repository
      |     metadata / SQL lineage / reconciliation
      |
```

```
        +-- Azure Databricks (SDK / OAuth)
            +-- Unity Catalog metadata
            +-- SQL Warehouse: table/column lineage
            +-- SQL Warehouse: grain SELECT
        |
        v
Structured response -> Claude technical report
```

3. Python code organization

Path / module	Responsibility
server.py	MCP adapter, tool registration and invocation
dwh_mcp/database.py	PostgreSQL metadata access
dwh_mcp/sql_repository.py	Repository SQL discovery and file reading
dwh_mcp/lineage.py	SQL lineage extraction
dwh_mcp/reconciliation.py	Mapping / constraint comparison
dwh_mcp/reporting.py	Report composition
dwh_mcp/composition.py	Legacy DWH object composition
dwh_mcp/models.py; errors.py	Models and error classification
dwh_mcp/databricks_provider.py	Unity Catalog metadata provider
dwh_mcp/databricks_lineage_provider.py	Statement Execution lineage queries
dwh_mcp/databricks_knowledge.py	On-demand knowledge composition
dwh_mcp/databricks_reconciliation.py	Identifier validation, SQL builder and grain interpretation
dwh_mcp/databricks_grain_provider.py	Exact grain SELECT via Databricks SQL Warehouse

4. MCP tool inventory and functional contracts

Exactly 21 registered tool names were observed in MCP client regression checks: 13 PostgreSQL/SQL/DWH and 8 Databricks tools. Parameter names are shown only where verified in the project sessions. Other tools should be documented from their current decorated Python definitions before public release.

4.1 PostgreSQL / SQL / DWH (13)

MCP tool	Purpose
list_schemas	List PostgreSQL schemas
list_tables	List tables in a schema
describe_table	Read table columns and metadata
get_primary_keys	Inspect primary key definitions
get_foreign_keys	Inspect foreign key relationships
list_sql_files	Discover SQL files in the repository
read_sql_file	Read a selected SQL definition
get_table_knowledge	Gather available table knowledge
get_table_lineage	Extract lineage from SQL definitions
reconcile_table_mapping	Compare declared table mappings
reconcile_column_constraints	Compare column-level constraints
generate_reconciliation_report	Prepare a reconciliation report
investigate_dwh_object	Composite DWH investigation

4.2 Azure Databricks (8)

MCP tool	Purpose
databricks_list_catalogs	List Unity Catalog catalogs
databricks_list_schemas	List Unity Catalog schemas
databricks_list_tables	List Databricks tables
databricks_describe_table	Retrieve Databricks table metadata
databricks_get_table_lineage	Read table-level lineage events
databricks_get_column_lineage	Read column-level lineage events
databricks_investigate_object	Compose an on-demand Knowledge Object
databricks_validate_grain	Execute exact key/grain validation

5. Verified Databricks tool invocation examples

The JSON objects below illustrate confirmed input and response structure; they are shortened and do not reproduce full live payloads.

5.1 databricks_investigate_object

Input: full_name (three-part catalog.schema.table string). Internally calls describe_table, get_table_lineage and get_column_lineage. The result combines metadata, table lineage, column lineage, provenance, limitations and status.


```
{
 "object_name": "marketplace_lab.gold.fact_orders",
 "platform": "azure_databricks",
 "metadata": {"full_name": "...", "table_type": "MANAGED"},
 "table_lineage": {"status": "completed", "events": ["..."]},
 "column_lineage": {"status": "completed", "events": ["..."]},
 "execution_context": {},
 "evidence": {"metadata_source": "unity_catalog"},
 "limitations": ["Lineage coverage is bounded"],
 "status": "completed"
}
```


5.2 databricks_validate_grain

Input: full_name and key_column. The provider builds a read-only aggregate SQL statement, executes it and computes duplicate excess as total_rows - distinct_keys - null_keys. Status is passed only when duplicate excess and NULL count are both zero.

```
INPUT
{"full_name": "marketplace_lab.gold.fact_orders", "key_column": "order_id"}

OUTPUT (validated result)
{
 "table_name": "marketplace_lab.gold.fact_orders",
 "key_column": "order_id",
 "total_rows": 10000,
 "distinct_keys": 10000,
 "duplicate_rows": 0,
 "null_keys": 0,
 "status": "passed"
}
```

Read-only SQL / SQL только на чтение

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT `order_id`) AS distinct_keys,
  SUM(CASE WHEN `order_id` IS NULL THEN 1 ELSE 0 END) AS null_keys
FROM `marketplace_lab`.`gold`.`fact_orders`
```

6. Data integrity, lineage and limitations

• Unity Catalog table statistics can be approximate; do not infer duplicates from statistics alone. Exact fact_orders validation reported 10,000 rows and 10,000 distinct order_id values, with no NULL keys and no duplicate excess.

• Databricks lineage queries read system.access.table_lineage and system.access.column_lineage; current query logic looks back 45 days and returns at most 50 events per call. Exactly 50 rows with truncated=false does not prove global lineage completeness.

• An event with source and target tables supports a direct lineage edge. An event with no target may represent a read rather than a downstream dependency. Column lineage does not supply SQL transformation expressions.

• execution_context in Knowledge Object v1 is empty. Event records can still carry entity_type/entity_id; resolving job names, notebook paths and SQL text is a future enhancement.

• The grain validator supports one simple key column and a simple three-part table identifier, not composite-key validation or exhaustive cross-platform semantic reconciliation.

7. Error handling, test boundaries and evidence

Validation area	Observed evidence
Knowledge composition	Mock provider tests, real Databricks test and Claude Desktop E2E passed.
Grain validation	SQL builder, mock SDK, real Warehouse SELECT, MCP client and Claude Desktop E2E passed.
Registry / compatibility	21 tools enumerated; PostgreSQL/DWH and Databricks groups co-registered.
Regression	Seven recent tests and five legacy tests passed; cross-platform registry and negative tests passed.
Error propagation	A simulated RuntimeError is surfaced to MCP Client with is_error=True.

Test scope note: mock-based tests do not prove remote-service availability. Real Databricks validation was checked separately. PostgreSQL/SQL integration has existing legacy coverage, but this document is not an exhaustive security or load-test certification.

8. Implementation notes for maintainers

• Keep 21 registered tool names stable unless making a documented API change. Registry tests should assert required names instead of relying only on exact counts.

• Use existing Databricks CLI/OAuth profile MARKETPLACE_LAB for local development. Never commit token caches, .env secrets, private configuration or machine-specific credentials.

• Explicitly state that SQL Warehouse requests can consume paid compute. For automated tests, mock providers unless a deliberately authorized integration test is being run.

• Before public release, verify actual source code, dependencies and current method signatures, produce configuration templates and perform a full secrets audit.

Source baseline: Stage 6.9.1 Architecture EN/RU and verified Stage 5–6 test outputs.